

Building AI Application Challenge 2026

Builder Progress Log & Submission Workbook

Days 2–8 (Environment → Build → Evaluate → Deploy → Submit)

0) Participant & Project Metadata

Field	Value
Participant Name	Rana Hamad
Email	Rana.hamad1@outlook.com
Timezone	GST (Dubai)
Start Date	2026-09-06
Path	■ LLM/API Integration
Team Members	Solo project
Mentor / Reviewer	None assigned yet
Project Repo URL	https://github.com/ranasalahe/GroupMate (private)
Demo / App URL	Pending — Hugging Face Spaces deployment is Day 7

Quick links: - ■ GitHub repo (private link above) - ■ Deployment link (Hugging Face) — pending Day 7 - ■ Demo video link — pending Day 8 - ■ LinkedIn progress post URL — pending - ■ Tweet/X progress post URL — N/A

1) Day 1 Idea Summary

Field	Value
Project Title	GroupMate
Concept Summary	An AI-powered task distribution app that fixes the most painful part of student group projects: figuring out who does what fairly. Members submit their strong suits, and the AI automatically splits project tasks based on skill fit and workload balance, then tracks progress, flags anyone falling behind, and keeps the whole group aligned in one place.
Target Audience	University and school students working on group projects, starting with students at Heriot-Watt Dubai and expanding to other student communities.

Field	Value
Key Features	1) AI-generated skill tags from the project description, with members selecting their strong suits. 2) AI-powered fair task distribution based on skill match and workload balance, including logic to keep large tasks whole vs. split as shared tasks. 3) Live group and individual progress tracking with progress rings and a deadline countdown. 4) Automatic falling-behind flagging based on task completion. 5) (Roadmap) Group chat, help-a-groupmate flow, personalized AI weekly schedules, and a private AI chat for guidance and conflict resolution.
Technical Approach	Python web app using Gradio for the interface, deployed on Hugging Face Spaces. Supabase serves as the backend database for groups, members, and tasks. Two sequential OpenAI API calls power the core intelligence: one generates project-specific skill tags from the project description, and a second distributes tasks fairly across members once everyone has submitted their tags, based on skill alignment and workload balance.
Expected Challenges	Getting the task-distribution prompt to produce consistently fair, well-reasoned splits across different project types; keeping the AI's JSON output reliably structured; scoping the build to a working core loop within the 8-day timeline.
Expected Outcome (by Day 8)	A working, deployed MVP covering group creation, AI-driven strong-suits tagging, AI task distribution, live progress tracking, and a falling-behind flag — demoable end-to-end, with remaining features (chat, help flow, personalized scheduling) presented as a clear product roadmap.
Additional Notes	Full product spec (including roadmap features) was mapped out in detail ahead of the build to keep scope decisions intentional rather than reactive.

2) Architecture Snapshot

Field	Value
One-line problem statement	Group project tasks get split unevenly and no one knows who's falling behind until it's too late.

Field	Value
Primary user story / flow	A student creates a group and adds project details and members → each member selects strong-suit tags generated by AI from the project description → once everyone has submitted, the AI distributes tasks fairly → members track and check off their tasks on a shared dashboard with progress rings, a group progress bar, and a countdown to the deadline.
Inputs	Project name, description, deadline; member names; member-selected strong-suit tags.
Outputs	AI-generated skill tag list; AI-distributed task assignments (including shared-task flags); per-member and group progress percentages; falling-behind flags.
Key components	UI: Gradio (5-tab <code>gr.Blocks</code> app in <code>app.py</code>). Backend logic: Python. Storage: Supabase (<code>groups</code> , <code>members</code> , <code>tasks</code> tables — see <code>supabase_schema.sql</code>). AI: OpenAI API (<code>gpt-4o</code> , JSON response mode) for tag generation and task distribution. Deployment: Hugging Face Spaces (planned, Day 7).
Data flow diagram	Create Group → <code>generate_tags()</code> (OpenAI) → Supabase <code>groups</code> row → Add Members → Supabase <code>members</code> rows → Strong Suits submitted per member → once all <code>submitted=true</code> , <code>distribute_tasks()</code> calls <code>ai_distribute_tasks()</code> (OpenAI) → Supabase <code>tasks</code> rows → Dashboard/My Tasks read from <code>tasks+members</code> and write back <code>completed</code> state.
Non-functional requirements	Latency: task distribution should return within a few seconds per call (single <code>gpt-4o</code> call, not yet benchmarked with real traffic). Cost: minimal at demo scale — 2 AI calls per group (tag generation + one-time task distribution). Privacy: no PII beyond member names and task text. Uptime: Hugging Face Spaces free tier is sufficient for demo purposes.

3) Data / Knowledge Sources Inventory

GroupMate does not use a fixed external dataset or corpus — inputs are provided live by users (project descriptions, member names, and strong-suit selections) and passed directly to the OpenAI API. This is not a RAG-based application, so the RAG-specific checklist is not applicable.

- Corpus assembled — N/A, not a RAG application
- Chunking strategy defined — N/A
- Embedding model selected — N/A
- Vector store selected — N/A
- Citation strategy — N/A
- Data safety: no secrets committed (`.env` gitignored, `.env.example` provided); no PII collected beyond member names

4) LLM / Model Selection & Experiments

Field	Value
Candidate	OpenAI gpt-4o
Provider	OpenAI
Why considered	Required by the challenge; strong structured-output (JSON mode) support, which both AI calls depend on.
Prompting approach	System + user message pattern, <code>response_format={"type": "json_object"}</code> to force valid JSON, with a manual retry (<code>_call_openai_json</code> , up to 2 retries) if a response fails to parse.
Quality notes	Live-tested on 2026-09-09 with a real key: <code>generate_tags()</code> returned 10 specific, relevant tags (e.g. "payment integration", "push notifications") for a bill-splitting app description on the first call, no retry needed. <code>ai_distribute_tasks()</code> split a 6-task project evenly across 2 members (3 tasks / ~30h each) with names matching exactly. Full <code>scripts/run_eval.py</code> (12-sample set) not yet run — see Day 4 testing notes below.
Latency notes	Each gpt-4o call (tag generation, task distribution, and the Day 4 "Ask AI" assistant) returns in roughly 2-4 seconds in manual testing — acceptable for this app's turn-based (not real-time chat) usage pattern.
Cost notes	Low at demo scale — one tag-generation call and one distribution call per group, plus occasional Ask AI calls; a few cents total across all manual testing so far.
Decision	gpt-4o via OpenAI API, JSON mode for structured calls, plain text for the Day 4 assistant, as required by the challenge.

Prompt versions

P1 — Tag Generation - Goal: Generate 6-10 project-specific skill tags from a project description. - System constraints: Respond ONLY in valid JSON `{"tags": [...]}`; tags specific to the project, not generic soft skills. - Few-shot examples: No — zero-shot with a tightly scoped system prompt. - Implementation: `generate_tags()` in `app.py`. - Notes / Results: Live-tested — returned 10 specific tags on the first try, no malformed-JSON retries triggered.

P2 — Task Distribution - Goal: Generate a task list from the project description and distribute it fairly across members by skill-tag match and workload balance, flagging shared tasks where splitting is more efficient. - System constraints: Respond ONLY in valid JSON matching `{"tasks": [{"title", "estimated_hours", "assigned_to", "shared", "shared_with"}]}`; every name used must exactly match a given member name. - Few-shot examples: No — zero-shot with a detailed system prompt and structured input (member names + strong suits). - Implementation: `ai_distribute_tasks()` in `app.py`. - Notes / Results: Live-tested — split a project into 6 tasks across 2 members, 3

tasks (~30h) each, all member names matched exactly on the first attempt.

Model-building path: Not applicable — GroupMate follows the LLM/API Integration path.

Day 2 — Environment Setup & Initial Development

Checklist - ■ Repository created and initial commit pushed - ■ README created (project goal + how to run) - ■ .gitignore configured (no secrets — .env, venv/, __pycache__/ excluded) - ■ Python environment created (venv, Python 3.14.6) - ■ Dependencies installed and pinned (requirements.txt) - ■ API keys stored safely (real .env created locally with OpenAI + Supabase keys; confirmed never staged in git) - ■ Basic 'hello world' run completed locally (Gradio Blocks builds successfully — verified via python -c "import app") - ■ Basic API call tested — live generate_tags() call against real OpenAI key succeeded on 2026-09-09 - ■ Folder structure created (app.py, docs/, scripts/, supabase_schema.sql) - ■ First prototype screen/flow created (5-tab Gradio UI: Create Group, Add Members, Strong Suits, Dashboard, My Tasks)

Evidence

Field	Value
GitHub repo URL	https://github.com/ranasalahe/GroupMate
Commit hash	b5d6a84 (initial scaffold), 243eb39 (hardening + eval set)
Screenshot(s)	Pending — to be captured once run with real credentials
Notes on setup decisions	OpenAI API key and Supabase project created ahead of time (schema pre-designed); Hugging Face account created for deployment. Environment set up using the py launcher (Python 3.14.6) since the plain python command hit Windows' App Execution Alias; dependencies pinned to exact installed versions (gradio==6.26.0, supabase==2.31.0, openai==3.8.0, python-dotenv==1.2.3) rather than a full pip freeze, to avoid pulling in Windows-only transitive packages that would break on Hugging Face's Linux runtime.

Initial development log - Set up the local Python environment (venv) and installed pinned dependencies. - Initialized the git repository, wrote the README, .gitignore, and .env.example. - Wrote the Supabase schema (groups/members/tasks) as a runnable SQL file. - Wrote the full app.py: Gradio 5-tab UI wired to Supabase and the two OpenAI calls. - Created a private GitHub repo and pushed the initial commit.

Blockers / issues encountered - Did not have laptop access on Day 1 — used that time to finalize the product spec, database schema, and AI prompts instead, and pre-wrote app.py so setup could move quickly once back on the laptop. Resolved on Day 2 with laptop access. - python was not recognized (Windows Store alias) — resolved by using the py launcher instead. - No GitHub CLI (gh) installed and Claude in Chrome extension not connected — resolved by creating the repo manually via the GitHub web UI and pushing over HTTPS from the command line.

Day 3 — Building the Brain of Your App

Checklist - ■ Core logic implemented (generate_tags(), ai_distribute_tasks(), distribute_tasks() in app.py) - ■ Data ingestion / API integration expanded beyond hello world (full Supabase CRUD across groups/members/tasks) - ■ 10-20 test questions/examples drafted (docs/test-project-descriptions.md — 12 samples spanning software,

research, creative, and event-planning projects, with group sizes from 1 to 6) - ■ First measurable baseline created (single-sample live test: 10/10 valid tags, 6/6 tasks correctly matched and evenly split — full 12-sample `scripts/run_eval.py` run still pending) - ■ README updated with run instructions

Artifacts / notes

Field	Value
What is the "brain" of your app	Sequential OpenAI API calls: one generates project-specific skill tags from the project description, a second distributes tasks fairly across members by matching skill tags to tasks while balancing overall workload (including logic for shared tasks), and (added Day 4) a third answers free-form teamwork questions using the project as context.
Link to key code	<code>app.py</code> — <code>generate_tags()</code> , <code>ai_distribute_tasks()</code> , <code>distribute_tasks()</code> , <code>ask_ai()</code>
Baseline results	Live single-run test (2026-09-09): tag generation 10/10 valid and relevant; task distribution 6/6 tasks assigned with names matching exactly and hours split evenly (3/3 tasks, ~30h each) across 2 members. Full 12-sample benchmark not yet run.
What to improve next	Run the full eval set for a larger sample size, tune the falling-behind threshold (<code>FALLING_BEHIND_THRESHOLD = 0.2</code>) against realistic usage patterns

Decisions made today (why) - Gated task distribution behind full strong-suits submission from all members (`submit_strong_suits` checks `all(m["submitted"])`), so distribution always runs against complete skill data. - Chose `gpt-4o` with JSON response mode for both AI calls to get reliably parseable output. - Matched members to AI-assigned tasks by exact name string rather than a member-ID lookup table in the prompt, to keep the prompt simple — traded off against needing the model to reproduce names exactly (mitigated by lowercasing/trimming on match, and the retry wrapper added Day 4).

Blockers / help needed - None — pending first live test run against real credentials.

Day 4 — Optimizing API Integration & Application Evaluation

Checklist - ■ Evaluation approach defined (metrics + test set) - ■ Error handling + retries added (API) - ■ Prompt/model iteration based on failures - ■ Latency/cost notes captured - ■ Safety/guardrails considered

APIs integrated and testing process

GroupMate integrates two external services: the **OpenAI API** (`gpt-4o`) and **Supabase** (Postgres + Storage), both used live (not mocked) as of today.

- **OpenAI, 3 call sites:** `generate_tags()` (JSON mode — project-specific skill tags), `ai_distribute_tasks()` (JSON mode — fair task splitting by skill + workload), and `ask_ai()` (plain text — an in-app assistant that answers teamwork/scheduling questions using the group's project description as context).
- **Supabase, 6 tables:** `groups`, `members`, `tasks`, `files`, `messages`, `help_requests`, plus a `group-files` Storage bucket for uploaded work.
- **Testing process:** manual, live, end-to-end. Real credentials were added to `.env` today, then tested directly (not just via the UI) with one-off Python calls to `generate_tags()` and `ai_distribute_tasks()`, followed by a full

click-through of the running Gradio app (create group → AI tags → add members → submit strong suits → automatic AI task distribution → dashboard → task checkoff → file upload/download → group and private chat → request/offer help → Ask AI). No automated test suite exists yet — testing is manual and live rather than CI-driven, which is a known gap for a longer-running project but reasonable for an 8-day challenge demo.

API rate limits & latency - Aware of OpenAI's per-tier rate limits (requests/tokens per minute on `gpt-4o`); GroupMate's usage pattern (a handful of calls per group action, not a chat loop) stays well under them at demo scale, so no request queuing or caching has been built yet — deliberately deferred rather than overlooked (see Day 6 risk log). - No real-time-interaction requirement (no streaming chat) means the observed 2-4s response time per call doesn't need special latency handling; the UI shows Gradio's built-in loading state while waiting.

Data handling and preprocessing - **Input:** project descriptions are `.strip()`-ed and capped at `MAX_DESCRIPTION_CHARS` (4000) before being sent to OpenAI; deadlines are parsed and validated (must be a real future date) before being stored or used in prompts. - **Output:** both structured AI responses are parsed as JSON (`json.loads`), then post-processed — task `assigned_to/shared_with` names are lowercased/trimmed and matched back to real member records before being written to Supabase, so the app never stores a raw, unvalidated AI string as a foreign key.

Fine-tuning - Not applicable at this scope. GroupMate uses `gpt-4o` off-the-shelf with prompt engineering (not fine-tuning) for both structured tasks — the task (generating tags / splitting work for an arbitrary, one-off project description) doesn't have a stable enough pattern across projects to justify building a fine-tuning dataset within the challenge timeline.

Performance & evaluation metrics - Generic LLM metrics like BLEU or sentiment scores don't fit this app (it's not translation or chat) — instead, evaluation uses task-specific metrics defined in `docs/test-project-descriptions.md` and `scripts/run_eval.py`: JSON/schema validity, name-matching correctness, and workload fairness (spread and stdev of hours across members). - The 12-sample eval set spans different project types and group sizes (1 to 6 members, including members with no strong suits selected) specifically to cover edge cases beyond the single live smoke test run so far.

Challenges or limitations faced - **OpenAI billing blocked initial testing** — the account had no credits, so the very first live call returned a `RateLimitError (insufficient_quota)`. Resolved by adding a small credit balance before any real testing could happen. - **JSON reliability** — `gpt-4o`'s JSON mode guarantees syntactically valid JSON but not schema conformance (e.g. a missing key). Mitigated with a retry wrapper rather than trusting a single attempt. -

Name-matching between AI output and database records — task distribution asks the model to reuse member names exactly; matching is done by lowercasing/trimming rather than IDs, which is simple but could silently drop a task if the model paraphrases a name. Not yet hit in testing, flagged as a watch item. - **No rate limiting or response caching implemented yet** — a user repeatedly clicking "Create Group" would trigger a fresh (billed) API call each time. Acceptable at demo scale, tracked as a Day 6 risk rather than blocking today's work.

Improvements/optimizations implemented today - Added a shared `_call_openai_json()` retry wrapper (up to 2 retries) instead of duplicating retry logic in both `generate_tags()` and `ai_distribute_tasks()` — one fix for one failure mode. - Wrapped every Supabase/OpenAI-touching function in `try/except`, returning a friendly message instead of crashing the whole Gradio app on a network or API error. - Added two input guardrails: a `MAX_DESCRIPTION_CHARS` (4000) cap on project descriptions before they're sent to OpenAI, and a check that the deadline is in the future. No HTML/injection sanitization was needed since Gradio renders text as plain text, not HTML. - Extended the AI surface from 2 to 3 integration points by adding the Ask AI assistant, reusing the same OpenAI client and error-handling pattern already in place.

Blockers / help needed - None currently — the full 12-sample `scripts/run_eval.py` benchmark (broader fairness testing across project types) is the next evaluation step, time permitting before Day 8.

Day 5 — Integration of Model/API with Interface

Checklist - ■ User flow designed (screens + inputs/outputs) - ■ Interface connected to backend/logic - ■ UX basics: error states (via Day 4 hardening); loading state covered by Gradio's built-in per-component spinners; explicit

reset/clear buttons not yet added - ■ Logging of inputs/outputs — not yet added - ■ Demo link — pending Hugging Face deployment (Day 7)

Artifacts / notes

Field	Value
Interface tech	Gradio (<code>gr.Blocks</code> , 5 tabs)
Demo link	Pending
Screenshots	Pending
Known UX issues to fix	Manual Group ID / name entry rather than join codes or authentication — acceptable for demo, flagged as a roadmap item. No reset/clear buttons on forms yet. No output logging yet (would need to avoid logging full project descriptions in plaintext beyond what's already stored in Supabase).

Decisions made today (why) - Built the interface across 5 Gradio tabs (Create Group, Add Members, Strong Suits, Dashboard, My Tasks) to keep the core loop simple and demoable within the timeline, matching the Day 1 user-story flow one tab at a time.

Blockers / help needed - None currently.

Day 6 — Final Enhancements, Security & Debugging

Checklist - ■ Input validation + sanitization — partial (description length cap, future-deadline check); full pass pending real testing - ■ Secrets handling reviewed (no keys in repo — confirmed `.env` and `venv/` are gitignored and were never staged) - ■ Rate limits / caching — not implemented, tracked as a risk - ■ Bug list triaged — pending real test run - ■ README + architecture notes cleaned — README is current; will revisit after real testing surfaces gaps

Artifacts / notes

Field	Value
Security checklist notes	API keys (OpenAI, Supabase) are read from environment variables / Hugging Face Space secrets, never hardcoded in <code>app.py</code> . Verified <code>.env</code> was never committed (<code>.gitignore</code> covers it from the first commit).
Top bugs fixed	N/A yet — pending real test run
Remaining risks	Supabase schema (<code>supabase_schema.sql</code>) uses open row-level-security policies (<code>using (true)</code>) for demo simplicity — anyone with the anon/service key can read/write any group's data. Acceptable for a short-lived challenge demo with no sensitive data, but flagged as the top item to tighten post-challenge (scope policies to a group-membership check, or require a Supabase Auth session per member). No rate limiting on the OpenAI calls — a user mashing "Create Group" repeatedly would rack up API cost with no cap.

Decisions made today (why) - Left RLS open rather than implementing per-group auth, given the challenge timeline and that the demo has no sensitive personal data beyond first names — documented as the main follow-up rather than silently shipping it.

Blockers / help needed - None currently.

Day 7 — Final Review & Deployment

Checklist - ■ Deployment target chosen and deployed — Hugging Face Spaces (Gradio SDK) chosen; not yet deployed - ■ Environment variables set in deployment platform - ■ Smoke tests run on deployed version - ■ Performance checked (latency/cost) - ■ Submission package checklist started

Artifacts / notes

Field	Value
Deployment platform	Hugging Face Spaces (Gradio SDK)
Deployment URL	Pending
Smoke test results	Pending
Fallback plan if deployment breaks	Present the working local version via screen recording/demo video if the live Space has issues.

Decisions made today (why) - Not yet reached — deployment requires logging into the user's Hugging Face account, which is done by the user directly rather than by an assistant (same boundary as the GitHub push in Day 2: repo/Space creation happens in the user's own authenticated session).

Blockers / help needed - Needs the user to create the Hugging Face Space (`README/app.py/requirements.txt` are already Spaces-compatible) and add `OPENAI_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY` as Space secrets.

Day 8 — Final Submission & LinkedIn Sharing

Checklist - ■ Final app link working - ■ Final repo is clean + documented - ■ Demo video recorded - ■ Final submission form completed - ■ LinkedIn post published + link shared

Artifacts / notes

Field	Value
Final app URL	Pending
Final repo URL	https://github.com/ranasalahe/GroupMate
Demo video URL	Pending
LinkedIn post URL	Pending
Tweet/X URL	N/A
Reflection: what you learned	To be filled in at the end of the challenge

Decisions made today (why) - Not yet reached.

Blockers / help needed - Deployment (Day 7) needs to land first; demo video and LinkedIn post depend on having a live link to show.

10) Community Sharing Tracker (bonus points)

Date	Platform	Post URL	What you shared	Tagged DDS?	Points claimed
Pending	LinkedIn	Pending	Progress update on GroupMate — AI-powered task distribution app for student group projects	Pending	Pending

12) Reviewer / Mentor Notes (optional)

Strengths observed: - *(left for reviewer)*

Areas to improve next: - *(left for reviewer)*

Action items: - *(left for reviewer)*